

Legacy Application Knowledge Extractor — come funziona

Due parti. La **prima** si legge senza sapere niente di programmazione: spiega a cosa serve l'applicazione e cosa fa, dall'inizio alla fine. La **seconda** è per chi ci deve mettere le mani: spiega i meccanismi, file per file.

PARTE 1 · In parole semplici

Il problema che risolve

In quasi tutte le aziende grandi c'è un programma vecchio che continua a funzionare e che nessuno sa più spiegare. È stato scritto vent'anni fa, chi l'ha scritto è andato in pensione, la documentazione non esiste — o esiste ma descrive una versione che non c'è più. Nel frattempo quel programma calcola gli sconti, emette le fatture, decide quali pratiche si bloccano.

Quando arriva il momento di rifarlo, di spostarlo sul cloud o semplicemente di capire perché fa una certa cosa, ci si trova davanti a un muro: **l'unica documentazione che dice la verità è il codice**, e il codice lo sanno leggere in pochi. Il lavoro tradizionale è mandare uno o due analisti a leggerlo per settimane, prendendo appunti.

Questa applicazione fa quel primo lavoro di lettura, in qualche minuto, e lo consegna in un formato che un esperto può correggere e firmare.

Cosa fa, in una frase

Le si dà del codice sorgente. Lei lo legge, ne ricava un documento che spiega cosa fa quel programma, quali regole di business contiene, con che altri sistemi parla, quali rischi si porta dietro — e lo consegna in Word, PDF o dati grezzi.

Come lo fa: due lettori diversi

Il punto interessante è che il codice viene letto **due volte, da due lettori con caratteri opposti**.

Il primo lettore è meccanico. Non capisce niente di quello che legge, ma non sbaglia mai. Sa riconoscere per certo che in quel file c'è una procedura che si chiama `CALC_SCONTO`, che tocca la tabella `ORDINI`, che c'è una password scritta a chiare lettere dentro il codice (cosa che non si dovrebbe mai fare) e che una certa istruzione può cancellare dei dati. Sono **fatti**: o ci sono o non ci sono, e non c'è opinione che tenga.

Il secondo lettore è un modello di intelligenza artificiale. Capisce il senso, ma può sbagliare. Sa dire che quelle trenta righe, messe insieme, significano «se l'ordine supera i cento euro si applica lo sconto del dieci per cento, tranne per i clienti esteri». È quello che serve davvero: il primo lettore ti dà l'elenco dei pezzi, il secondo ti dice a cosa servono.

L'applicazione fa parlare i due lettori tra loro. **Al modello vengono passati i fatti del lettore meccanico insieme al codice**, con l'istruzione esplicita di non contraddirli. Poi i due risultati vengono uniti in un'unica tabella, dove per ogni riga è scritto **chi l'ha trovata e quanto è sicura**.

Questo è il cuore di tutto: un'informazione che viene dal lettore meccanico ha un valore diverso da una che viene dal modello, e nel documento finale la differenza resta visibile. Non si mescolano i fatti con le interpretazioni.

Come si legge quello che trova

Ogni riga porta scritto da dove viene e quanto è solida, e lo dice con una forma oltre che con un colore: un quadrato pieno è un fatto del lettore meccanico, che su quello non può sbagliare; un quadrato vuoto è la lettura del modello, con accanto tre pallini che dicono quanto ci crede. La gravità dei rischi è l'unica cosa calda della pagina, così l'occhio ci va per primo. Niente è affidato al solo colore: chi non distingue il rosso dal verde, e chi stampa in bianco e nero, legge le stesse identiche informazioni.

Il terzo lettore: la persona

Il documento non è finito quando l'applicazione ha finito. Ogni cosa trovata finisce in una tabella con **una casella da spuntare**: è lì che un esperto di quel dominio — chi conosce il business, non chi conosce il codice — passa riga per riga e conferma, corregge o cancella.

L'applicazione lo aiuta scrivendo anche una lista di **domande a cui il codice non sa rispondere**. Per esempio: nel codice c'è scritto «trenta giorni», ma *perché* trenta? È un termine di legge, un accordo con un fornitore, o un numero che qualcuno ha scritto nel 2003 e nessuno ha più toccato? Quella risposta non sta nel codice, sta nella testa di qualcuno. L'applicazione sa di non saperlo, e lo chiede invece di inventare.

Il giro completo

1. **Si caricano i file.** Codice Oracle, COBOL, Visual Basic, Java, Python e qualunque altro file di testo: se l'estensione non è fra le ottanta conosciute, entra lo stesso e il modello capisce il linguaggio dal contenuto. Solo i file binari vengono respinti. Oppure si incolla direttamente un pezzo di codice.
2. **L'applicazione legge meccanicamente.** In pochi secondi, senza costi.
3. **Manda tutto al modello.** Se il codice è tanto, lo divide in lotti e ne manda uno alla volta, senza mai spezzare un file a metà.
4. **Rimette insieme e ripulisce.** Toglie i doppioni, numera le righe, raddrizza quello che il modello ha scritto male.
5. **Mostra tutto in schede:** sintesi, regole di business, architettura, flussi di dati, rischi, diagrammi, domande per l'esperto.
6. **L'esperto valida.** Spunta, corregge, cancella.
7. **Si esporta** in Word, PDF o dati grezzi. Il file dei dati grezzi si può ricaricare in un altro momento per riprendere la validazione da dove era.

Quanto ci mette

Il tempo lo fa la risposta, non la domanda: il modello scrive qualche decina di parole al secondo, e una risposta completa sono minuti. Un'analisi vera dura minuti, non secondi — si stanno leggendo decine di migliaia di righe — e la barra di avanzamento mostra i secondi che passano e i lotti finiti. Quattro manopole per accorciarla, tutte nella barra laterale:

- **Profondità.** «Quick» chiede solo le sezioni che ripagano il lavoro (processi, regole, componenti, dipendenze, interfacce, dati, rischi, domande) e una risposta lunga meno della metà: grosso modo metà tempo. «Full» chiede tutto. I diagrammi si costruiscono comunque dalle tabelle.
- **Quanto si lascia pensare il modello** prima che risponda (*Reasoning effort*). Lo stesso modello, con il ragionamento al massimo, impiega minuti dove al minimo impiega secondi. L'applicazione adatta la scelta da sola: alcuni modelli non accettano i livelli più bassi, e in quel caso sale di un gradino invece di fermarsi con un errore.
- **Lotti insieme.** Quando il codice è tanto e viene diviso in lotti, se ne possono mandare al modello fino a quattro alla volta. Più veloce, ma consuma la quota più in fretta: con una chiave gratuita conviene restare a uno o due.
- **Memoria del lavoro fatto.** Un lotto già analizzato — stessi file, stesse impostazioni — non si ripaga: si rilegge dal disco, anche il giorno dopo, anche da un collega sulla stessa macchina. Aggiungere un file costa solo quel file. «Analyse again from scratch» la ignora.

Quando il modello si ferma a metà

Succede, quando la risposta è lunga: il modello arriva al tetto di parole e si ferma prima della fine. L'applicazione lo sa da un segnale che il modello mette in fondo alla risposta solo quando ha davvero finito. Se manca, gli chiede di **continuare dal punto esatto** in cui si è fermato, con lo stesso modello — mai con un altro, che ricomincerebbe da capo — fino a due volte da sola.

Se non basta, in cima alla pagina compaiono tre scelte, con scritto quali lotti e quali file sono a metà: *Continue with the same model*; *Keep what was written*, che tiene le righe già scritte e chiude lì; *Discard and start over*, che butta tutto e riaccende l'avvio. Se il modello che scriveva ha smesso di rispondere — quota finita, servizio giù — compare anche *Continue with the next model*: il seguito passa al modello successivo, e il documento dirà che quel lotto l'hanno finito in due. L'applicazione non cambia mai modello da sola a metà di una risposta; ma non lascia nemmeno nessuno fermo. Finché la risposta non è completa non si può chiederne una nuova: prima si finisce, in uno di questi modi, poi eventualmente si rifà.

Interrompere e riprendere

Il lavoro di validazione dura giorni, e una scheda del browser no. Il JSON che si scarica dalla scheda *Export* contiene tutto — righe, spunte, correzioni, chi ha risposto — e per riprendere basta **trascinarlo nel campo dei file** insieme, o al posto, dei sorgenti: l'applicazione lo riconosce dal contenuto e riparte da dove si era. Vale anche per i file salvati con le versioni precedenti dell'applicazione.

Se il sorgente è andato perso

Si può caricare direttamente l'eseguibile: l'applicazione lo esamina e ne ricava un resoconto — com'è stato costruito, contro quali librerie è linkato, che stringhe contiene (query SQL, connection string, percorsi, messaggi) — che poi viene analizzato come se fosse codice. E se il sorgente è stato ricostruito con un decompilatore, l'applicazione **se ne accorge da sola** e si comporta di conseguenza: i nomi che un decompilatore inventa (`FUN_00401a30`, `uVar7`) non significano niente, quindi il modello viene avvertito di ignorarli e nessuna riga che nasce da quei file può dichiararsi certa. In copertina del documento è scritto quanti file sono di questo tipo.

Quanto si recupera dipende da come il programma è stato compilato: da un .NET o da un Java il sorgente torna quasi per intero, da codice nativo si ottiene codice che fa la stessa cosa ma senza nomi né commenti.

Usare la chiave Azure aziendale

L'applicazione si apre già pronta per la risorsa Azure aziendale: provider **Microsoft Azure OpenAI**, l'endpoint della risorsa e il deployment **gpt-6-luna** sono già scritti. Manca solo la chiave.

Su Streamlit Cloud la chiave si mette una volta sola nei *Secrets* dell'applicazione (*Manage app* → *Settings* → *Secrets*):

```
AZURE_OPENAI_API_KEY = "la-chiave-aziendale"
```

Da quel momento l'applicazione si apre con tutto compilato: si carica il codice e si analizza. **In locale** la stessa cosa si fa con una variabile d'ambiente con lo stesso nome. **La chiave non va mai scritta nel codice né messa su git.**

Per controllare che tutto risponda c'è **Check the connection**, sotto la chiave. Per usare altri modelli oltre a gpt-6-luna si aggiungono nel campo **Deployments**, separati da virgola e nell'ordine in cui provarli — per esempio `gpt-6-luna`, `gpt-5.6-luna`: il primo lavora, gli altri sono la riserva. Si possono anche fissare nei *Secrets* con `AZURE_DEPLOYMENT`, e l'endpoint con `AZURE_OPENAI_ENDPOINT`, se un giorno cambiasse.

Quali modelli ci sono. Li decide chi gestisce la risorsa. Oggi, secondo lo snippet, `gpt-6-luna` e `gpt-5.6-luna` per il testo; `gpt-image-2` genera immagini e l'applicazione lo esclude da sola, perché non legge codice. Se la chiave ha il permesso di elencare i deployment, l'applicazione li trova da sé; altrimenti — il caso più comune con una chiave aziendale — si scrivono a mano. Per un modello nuovo va chiesto il deployment a chi gestisce la risorsa, e poi aggiunto all'elenco.

Quanto farlo ragionare (*Thinking time*, in *Expert settings*). Tre gradini: Fast è `low`, Balanced `medium`, Deep `high`. Balanced è il predefinito e il punto di partenza giusto per leggere codice legacy; su `gpt-5.6-luna`, che lo snippet indica per lo sforzo alto, conviene Deep. I livelli `xhigh` e `max` esistono ma l'applicazione non li offre: su una risposta lunga come la nostra vogliono dire minuti per ogni lotto. Il ragionamento consuma lo stesso tetto della risposta, e l'applicazione gli riserva un margine da sola.

Quanti lotti in parallelo (*Batches at once*). Su Azure il limite è **per deployment**, in token al minuto (TPM) e richieste al minuto: si legge nel portale Azure AI Foundry, alla voce del deployment, o lo si chiede a chi gestisce la risorsa. Conta che Azure, per decidere se accettare una richiesta, stima i token **prima** di eseguirla, compreso il tetto della risposta: un lotto pieno vale fra 50 000 e 70 000 token a seconda del ragionamento. A spanne: sotto i 100 000 TPM un lotto alla volta; fra 100 000 e 300 000, due; sopra, tre o quattro. Se si esagera non si rompe niente: il 429 di Azure viene aspettato e riprovato **sullo stesso modello**, costa solo tempo.

Cosa resta sulla risorsa. Ogni chiamata parte con `store: false`: la risposta non viene conservata sul server per essere ripresa, come invece fa la Responses API di serie. Il sorgente del cliente passa, non resta.

I diagrammi

Ci sono quattro disegni: come scorre un processo, con che sistemi esterni parla l'applicazione, dove vanno i dati, e chi chiama chi dentro il codice.

Non li disegna il modello — o meglio, se lui li produce vengono tenuti, ma **normalmente vengono costruiti dalle tabelle stesse**. È una scelta precisa: se il disegno nasce dalle tabelle, non può raccontare una cosa diversa da quello che c'è scritto nelle tabelle. E soprattutto, quando l'esperto corregge una riga, **il disegno si può rifare** e si aggiorna. Prima capitava che qualcuno cancellasse un collegamento sbagliato e il disegno continuasse a mostrarlo: il documento firmato conteneva due verità diverse.

I diagrammi vengono anche disegnati **sul computer di chi usa l'applicazione**, non mandati a un servizio su internet. Su codice di un cliente è una differenza che conta.

Cosa questa applicazione NON fa

Vale la pena essere netti, perché le aspettative sbagliate su questi strumenti sono la cosa che li fa fallire.

- **Non riscrive il programma** e non lo traduce in un altro linguaggio.
- **Non garantisce di aver trovato tutto.** Gli indicatori dicono quanto è solido quello che ha trovato — quanti file ha citato, quante righe hanno un riscontro nel codice — non quanta parte dell'applicazione ha capito. Nessuno strumento automatico può dire quest'ultima cosa.
- **Non sostituisce l'esperto.** Produce una bozza ragionata da correggere. Un documento uscito da qui e non validato da nessuno non vale più della bozza che è.
- **Non è un giudice.** Quando dice che una cosa è «rischiosa» sta segnalando un sospetto, con un livello di gravità; sta a chi legge decidere.

Perché è costruita così

Tre idee attraversano tutto il progetto, e conviene conoscerle perché spiegano quasi ogni scelta:

Un dato inventato è peggio di un dato mancante. Un buco lo si vede e lo si riempie. Una riga plausibile ma falsa entra nel documento, viene firmata, e qualcuno ci costruisce sopra un progetto di migrazione.

Chi legge deve sapere da dove viene ogni cosa. Fatto certo o interpretazione, macchina o modello, quanto sicuro: sempre scritto.

Un messaggio deve promettere solo quello che il codice farà davvero. Se qualcosa non funziona perché la chiave d'accesso è sbagliata, l'applicazione non dice «riprova più tardi» — perché riprovando non cambierà niente, e chi legge perderebbe la giornata. Dice che la chiave è sbagliata.

PARTE 2 · Per gli addetti ai lavori

Architettura

Applicazione Streamlit, otto moduli Python più tre script di servizio, nessun database, nessuno stato sul server oltre alla sessione.

File	Responsabilità
app.py	Pagina, analisi statica, orchestrazione, stato di sessione
ui.py	Tema, componenti e marcatori di provenienza dell'interfaccia
model_chain.py	Scoperta, prova e scalata dei modelli sui tre provider
contract.py	Il contratto JSON: prompt, schema, riparazione, normalizzazione
diagrams.py	I quattro diagrammi ricavati dai dati strutturati
mermaid_render.py	Mermaid → PNG, in locale (mmdc) o come ricaduta remota
exporter.py	PDF (ReportLab) e Word (python-docx): una descrizione, due rese
fonts/	IBM Plex per il PDF (licenza OFL, con il file di licenza accanto)
test_catena.py	227 casi, senza rete, senza chiavi, senza costi
avvia.py	Installa quello che manca, configura Streamlit e apre l'applicazione
binary_triage.py	Esame di un eseguibile di cui manca il sorgente
verifica.py	Controlla che un clone sia completo, senza toccare niente
guida_pdf.py	Rigenera le due guide (italiano e inglese) in PDF (servizio, non serve all'app)

Le dipendenze fra moduli vanno in una direzione sola: `app` → tutti; `exporter` → `mermaid_render`; `diagrams` → `contract`. Nessun ciclo, e `model_chain` non sa niente né di Streamlit né del dominio: si può usare da riga di comando o riportare in un altro progetto così com'è.



Fase 1 — Ingestione

`build_source_collection` accetta qualunque file, respinge i binari (un byte nullo nei primi 8 KB), decodifica provando UTF-8, UTF-8-BOM, CP1252, Latin-1 in quest'ordine (i sorgenti legacy sono quasi sempre CP1252 o EBCDIC convertito male), calcola uno SHA-256 troncato per file e riconosce il linguaggio dall'estensione — ottanta conosciute, le altre marcate «Unknown» e lasciate al modello. Tetti: 2 MB per file di testo, 50 MB per un eseguibile (il resoconto che ne esce ha comunque dimensione fissa), 1,5 M caratteri in totale.

Profondità. `PROFONDITA` in `contract.py` definisce cosa chiedere: «quick» otto sezioni e 7.000 token di risposta, «full» tutto e 16.000. Prompt, schema nativo e normalizzazione leggono la stessa struttura; le sezioni saltate restano vuote senza avvisi.

Lotti in parallelo e cache. `analyze_legacy_application` fa la prova di contatto una volta, poi manda i lotti a un `ThreadPoolExecutor` (1-4 thread a scelta); l'avanzamento si aggiorna solo dal thread principale. Prima di chiedere, ogni lotto cerca in `cache/` una risposta con la stessa chiave — impronte dei file, contratto, provider, preferenza, ragionamento, profondità — e la riusa; dopo, la scrive.

`split_into_batches` divide in lotti da ~120.000 caratteri **senza mai spezzare un file** (un file tagliato a metà produce regole di business monche, che è peggio di un file in meno) e **tenendo insieme i file che si chiamano fra loro**, presi dal grafo delle dipendenze del parser: due file legati finiti in lotti diversi non li guarda insieme nessuno.

Fase 2 — Analisi statica

Tutto in `app.py`, sezioni 5 e 6. Produce i **fatti**.

Parsing SQL (`parse_sql_expressions`): `sqlglot` con `error_level="ignore"`, provando in sequenza i dialetti `None`, `oracle`, `mysql`, `postgres`, `tsql` e tenendo il primo che restituisce un albero non vuoto. Dall'AST si estraggono tabelle (`exp.Table`), colonne (`exp.Column`), tipo di operazione e JOIN con la condizione.

Riconoscimento a espressioni regolari per quello che `sqlglot` non copre:

- componenti: `PROCEDURE`, `FUNCTION`, `PACKAGE`, metodi Java, `def` Python, `function` JavaScript, paragrafi COBOL, `Sub/Function/Property/Class` di Visual Basic, `sub` Perl, step JCL — **ogni pattern solo sul suo linguaggio**, tutti insieme solo se il linguaggio è ignoto;
- dipendenze dichiarate: `import`, `require`, `#include`, `COPY`, `/COPY`;
- chiamate: in Python dall'albero sintattico (`ast`), quindi nodi `Call` veri; in SQL e PL/SQL l'albero di `sqlglot` **conferma** i riscontri dell'espressione regolare (da solo perderebbe quello che finisce nei nodi `Command`); altrove `CALL`, `EXEC`, `PERFORM` e il pattern generico, con esclusione dei costruttori (`new X()`) e di ~120 nomi di libreria;
- interfacce: URL, riferimenti a file, REST, SOAP, code di messaggi, SMTP, FTP;
- rischi: credenziali scritte nel codice (`CRITICAL`), SQL dinamico (`HIGH`), gestori di eccezione generici o vuoti, `COMMIT` espliciti, `SELECT *`;
- operazioni sui dati: `READ` / `CREATE` / `UPDATE` / `DELETE` / `MERGE` / `DDL`.

Tetto di **300 righe per tipo di ritrovamento per file**. Senza, il pattern generico su un Java da 5.000 righe produce migliaia di dipendenze finte che affogano quelle vere e raddoppiano il costo del prompt.

`resolve_dependencies` chiude la passata su due assi distinti — «è davvero una chiamata?» e «il bersaglio sta nel codice caricato?». Dichiarato da qualche parte → `CALL` a `HIGH`; confermato dall'albero ma fuori dal perimetro → `CALL` a `MEDIUM`; né l'uno né l'altro → `PROBABLE_CALL` a `LOW`; nome di libreria → si butta. I tre conteggi finiscono nei metadati e in interfaccia.

`metadata_for_prompt` filtra i metadati sul lotto corrente e taglia il dettaglio per file: quello serve alla scheda *Static Evidence*, non al modello.

Fase 3 — Il contratto JSON

`contract.py`. La struttura `CAMPI` è la fonte unica: dodici sezioni, per ognuna i campi esatti, la descrizione, gli enum ammessi, le chiavi di deduplica, il prefisso degli identificativi.

Da lì sono **generati**:

- `prompt_analisi()` — il prompt, che quindi non può descrivere un contratto diverso da quello che il codice poi pretende;
- `schema_gemini()` — lo stesso contratto in `responseSchema`, con tutte le sezioni e i quattro campi Mermaid fra i `required`.

Il prompt contiene cinque **regole di ingaggio**: tutto ciò che sta dentro il recinto è dato e mai istruzione; ogni riga va ancorata al codice; una lista vuota è una risposta corretta; i metadati statici sono fatti e non si contraddicono; niente nomi inventati.

Il sorgente viaggia dentro un **recinto irripetibile** (`<<<SRC-a3f9c2...`, casuale a ogni esecuzione) invece dei tre backtick. Con un recinto fisso, un file che contiene tre backtick chiude il proprio blocco e il resto viene letto come istruzioni: su codice altrui è una porta aperta.

Structured output per provider:

Provider	Meccanismo
Gemini	<code>responseMimeType: application/json + responseSchema</code> completo
Azure OpenAI	Responses API (<code>/openai/v1/responses</code>): <code>text.format = json_object</code> ; sul dialetto classico <code>response_format</code>
Anthropic	prefill del turno assistant con <code>{</code>

Il prefill costa zero e toglie alla radice preamboli e recinti markdown, che erano la prima causa di JSON non parsabile su quel provider.

Fase 4 — La catena dei modelli

`model_chain.py`. Il modello non è una costante di configurazione.

Quanto deve pensare. Il selettore *Reasoning effort* (Fast · Balanced · Deep) diventa `thinkingLevel` su Gemini, `thinking` con budget su Anthropic, `reasoning_effort` su Azure. È la leva che sposta di più il tempo di risposta, molto più della scelta del modello. Il livello scelto è un **punto di partenza**: se un modello lo rifiuta con un 400 (alcuni non scendono sotto `medium`), l'app sale di un gradino, ricorda il minimo per quel modello e riprova — chi sceglie «Fast» ottiene il più veloce che quel modello sa fare, non un errore. Un adattamento non consuma i tentativi riservati ai guasti veri, e la prova di contatto resta sempre al livello più basso.

Il modello preferito. Il menù *Preferred model* nella barra laterale si riempie con i modelli che la scoperta trova sulla chiave; *Automatic* lascia decidere la catena, una voce specifica la mette in testa (`preferito` in `CatenaModelli`), con gli altri come rete e la scelta che vince sulla memoria.

Scoperta. `GET /v1beta/models` (Gemini), `GET /v1/models` (Anthropic), `GET /openai/deployments` (Azure). Si filtrano `-lite`, `preview`, `exp`, `latest`, i nomi con `data`, gli `embedding/tts/vision/live`; per Gemini si richiede versione ≥ 2.0 . Restano al massimo cinque modelli, ordinati per **fascia** poi per versione: con `Quality first` la catena parte da `pro/opus/gpt-5,03` e scende; con `Speed first` si ottiene la regola di Nuvia (solo i veloci). Su Azure il deployment scritto a mano resta sempre primo, perché il nome chiamabile lo conosce solo chi ha creato la risorsa. Se la scoperta fallisce si usa una catena scritta nel codice: è una rete, non un fallimento da mostrare.

Prova. A ogni modello, in ordine: «Rispondi con una sola parola: OK», tetto **5 s per modello** e **10 s complessivi**. Il primo che risponde vince. Un secondo tentativo sullo stesso modello solo su guasti transitori e solo se restano almeno altri 5 s: altrimenti la seconda chance è il modello dopo, che vale di più. Un 200 con testo vuoto conta come vivo (i modelli che ragionano possono consumare il tetto di token del test prima di scrivere).

Chiamata vera. Parte dal modello che ha risposto e scende **solo in giù**, mai risale: tre tentativi per modello con attesa crescente.

Tassonomia dei guasti, unica per i tre provider: `quota`, `modello`, `badkey`, `busy`, `timeout`, `rete`, `vuota`, `troncata`, `blocked`, `contesto`, `parametro`. Ogni provider traduce il proprio dialetto HTTP in questi nomi. Tre insiemi decidono il comportamento: `RIPROVA` (stesso modello), `CAMBIA` (modello successivo), `NON_SCENDERE` (inutile proseguire).

Il 400 che non è una chiave sbagliata. Prima di dare la colpa alla chiave si legge il corpo dell'errore: se parla di `schema`, `temperature`, `max_completion_tokens` o `thinking`, si adatta la richiesta, si **ricorda l'adattamento per quel modello** e si riprova. La volta dopo si parte già giusti. È lo stesso meccanismo del `thinkingLevel` di Nuvia.

Guardia a orologio. Ogni richiesta gira in un thread contro una scadenza vera, perché il `timeout` di `requests` è per singola lettura del socket: un server che manda un byte ogni tre secondi non lo fa scattare mai e il tetto dei cinque secondi smette di esistere.

Memoria. Modello buono 10 minuti, elenco scoperto 6 ore, adattamenti per sempre. Su file o in RAM. Della chiave si salva solo un'impronta SHA-256 troncata come identità della cache: due chiavi non si scambiano i dati e la chiave non finisce mai su disco.

I messaggi. «Riprova più tardi» solo se riprovare può cambiare qualcosa. Chiave sbagliata, quota esaurita, contesto pieno hanno il loro messaggio, in italiano e in inglese.

Il trasporto è `requests` diretto, non i tre SDK: gli SDK ritentano per conto loro *sopra* la nostra scalata (il client OpenAI ritenta i 429 di default, mangiandosi il tempo del nostro tetto) e nascondono lo status HTTP che la tassonomia usa per decidere.

Fase 5 — Rientro e normalizzazione

`estrai_json` toglie eventuali recinti, tenta `json.loads`, poi tenta il sottoinsieme fra la prima `{` e l'ultima `}`, poi chiama `ripara_troncato`.

`ripara_troncato` scorre il testo tracciando stringhe, sequenze di fuga e pila delle parentesi, torna all'ultimo confine sicuro e richiude quello che era aperto. **Il tetto di token è il modo più comune di perdere un'analisi da tre minuti**: prima l'ultima riga a metà faceva buttare anche le duecento righe perfette che venivano prima.

`normalizza` fa il resto: campi mancanti riempiti, nomi di campo simili recuperati, enum raddrizzati con una tabella di sinonimi («High», «critico», «Sev2»), liste convertite in testo (A; B), celle tagliate a 1.500 caratteri, righe vuote scartate, dopponi tolti sulle chiavi dichiarate, identificativi assegnati **dal codice** (il modello riparte da 1 a ogni lotto), Mermaid ripulito. Tutto quello che ha dovuto aggiustare finisce in `contract_warnings`, visibile in interfaccia: chi valida deve sapere quanto fidarsi.

La pulizia Mermaid merita una riga: le etichette vengono messe fra virgolette e ripulite da `( ) [ ] { } " ; |` con uno scanner scritto a mano, non con una regex, perché le forme si annidano (`A[Ordine (nuovo)]`) e una regex prende la parentesi interna lasciando fuori quella che conta. Lo stesso scanner sa distinguere il testo visibile dagli identificatori, e nei secondi rinomina le proprietà del prototipo JavaScript (`toString`, `valueOf`, `constructor`...) che altrimenti fanno morire il `render`. `end` e `subgraph` restano intatti: lì la parola ha un significato.

Consolidamento. Con più di un lotto parte una chiamata finale che riceve **solo l'inventario**, senza codice: produce una sintesi unica dell'applicazione e cerca i collegamenti fra lotti diversi. Ogni nome che torna viene verificato contro l'inventario — in quella chiamata il modello non ha il codice davanti, quindi un nome inventato non lo smentirebbe nessuno. Se fallisce, si tengono le sintesi per lotto e lo si dichiara.

Completezza e continuazione. Il contratto chiede che l'ultima proprietà sia `"complete": true`. `risposta_completa` la considera completa se il JSON si legge per intero e c'è il tag, o — senza tag — se il provider non l'ha segnalata come tagliata. Se non lo è, `CatenaModelli.continua` chiede il seguito **allo stesso modello** (su Anthropic il pezzo scritto è il prefill e il modello continua la stessa frase; su Gemini e Azure torna come turno precedente), `unisci_continuazione` lo riattacca tagliando la sovrapposizione, e si ricontrolla. Due giri automatici; poi le quattro scelte (stesso modello, modello successivo via `CatenaModelli.successivo` solo se il primo non risponde più, tenere il parziale, buttare tutto). Una continuazione automatica che fallisce non fa cadere l'esecuzione: il lotto resta a metà con la causa. Lo stato dei lotti — prompt, testo, modello — vive nella sessione e non nel JSON esportato, perché il prompt contiene il sorgente del cliente.

`unisci` somma i lotti sulle stesse chiavi di deduplica, concatena i testi, per i diagrammi tiene il più completo (due `flowchart TD` incollati non si disegnano) e rinumerava.

`merge_static_and_ai_results` fa passare **anche le righe del parser** dal contratto prima di unirle: è l'unico modo perché le tabelle abbiano le stesse colonne.

Fase 6 — I diagrammi

`diagrams.py` costruisce i quattro disegni dai dati strutturati: `call graph` da `dependencies`, `application map` da `application_mapping` (o `interfaces`), `data flow` da `data_flows` (o `data_objects`), `process flow` da `business_processes`. Si costruiscono **dopo** l'unione con l'analisi statica, così il `call graph` contiene anche le dipendenze trovate dal parser.

`arricchisci` sceglie: se il modello ha prodotto qualcosa di sostanzioso (≥ 3 righe) resta il suo, altrimenti si mostra quello costruito. Entrambe le versioni restano nel risultato e in interfaccia c'è l'interruttore, più un bottone che rifà i disegni dalle tabelle correnti — cioè dopo le correzioni dello SME. La provenienza si scrive accanto al disegno, **non** fra gli avvisi di contratto: costruire il diagramma dai dati è il funzionamento previsto, e nel pannello degli avvisi sembrava un guasto.

Garanzie per costruzione: id unici e sempre preceduti da `n_` — un id nudo può coincidere con una proprietà che ogni oggetto JavaScript eredita (`toLocaleString`, `constructor`), e Mermaid, che tiene i nodi in un oggetto normale, crede che il nodo esista già e muore con «Cannot set properties of undefined (setting 'order')»; etichette senza i caratteri che rompono mermaid; tetto di 40 archi con dichiarazione di quanti restano fuori; quando si taglia, escono per prime le `PROBABLE_CALL` e le confidenze basse.

Fase 7 — Rendering ed esportazione

La configurazione del disegno (tema, etichette SVG invece che HTML, larghezza di ritorno a capo) viene scritta **dentro** il diagramma come direttiva `%%{init: ...}%%`: un file di configurazione vale solo per il disegno locale, mentre il servizio esterno riceve soltanto il codice.

`mermaid_render.rendi` prova nell'ordine: `mmdc` locale (`@mermaid-js/mermaid-cli`, cioè `mermaid.js` dentro un Chromium headless), poi `npx` se esplicitamente abilitato, poi `mermaid.ink` come ricaduta. `MERMAID_LOCAL_ONLY=1` vieta del tutto l'uscita. Gestisce `--no-sandbox` per i container con un file di configurazione Puppeteer temporaneo, rispetta `PUPPETEER_EXECUTABLE_PATH`, e mette in cache sull'impronta del codice (PDF e Word chiedono gli stessi quattro diagrammi).

`exporter.py` prepara **una sola descrizione** del documento — copertina, titoli, prosa, tabelle, diagrammi — e la disegna due volte, in PDF (ReportLab, A4 orizzontale, IBM Plex) e in Word (python-docx). Prima erano due funzioni parallele che avevano già smesso di dire le stesse cose.

Le tabelle portano la provenienza di ogni riga (`parser` o `model`), la confidenza come puntini e il segno di conferma dell'esperto; i rischi escono ordinati per gravità e le dipendenze per affidabilità. Le immagini rispettano **le proporzioni lette dall'immagine stessa** (intestazione PNG o JPEG, altrimenti PIL); non esiste una misura fissa di ripiego.

Il documento ha otto sezioni: sintesi, logica di business, architettura, dati, rischi, diagrammi, **domande aperte e assunzioni**, e in appendice quello che ha trovato il parser.

Stato, cache e costi

Streamlit riesegue lo script intero a ogni interazione. Di conseguenza:

- il risultato vive in `st.session_state` e le modifiche delle tabelle ci rientrano a ogni giro; lo stato dei lotti (prompt, testo scritto finora, modello) sta in `st.session_state["lotti_stato"]`, separato dal risultato, perché il prompt contiene il sorgente del cliente e non deve finire nel JSON esportato;
- `analysis_signature` (impronta dei file + provider + profondità + versione del contratto) evita di rilanciare un'analisi identica alla precedente;
- la cartella `cache/` (visibile, esclusa da git, cancellabile) tiene i lotti già analizzati — chiave: impronte dei file, contratto, provider, preferenza, ragionamento, profondità — e la memoria della catena; solo i lotti completi ci finiscono;
- PDF e Word si generano **su richiesta** e sono in `st.cache_data`: prima ogni spunta su una casella rigenerava entrambi, scaricando otto immagini;
- il JSON esportato porta anche `_metadata` e chi ha risposto, e si ricarica con `carica_analisi_salvata`, che lo fa passare dal contratto come una risposta del modello (spunte comprese) e ricostruisce i diagrammi.

Collaudo

`python test_catena.py` — 227 casi, nessuna rete, nessuna chiave, nessun costo. Il provider è finto: si dichiara quali modelli rispondono e come, e si osserva il comportamento. Copre scoperta e ordinamento, scalata, riprova selettiva, memoria, messaggi nelle due lingue, tetti temporali, adattamento dei parametri, riparazione del JSON, enum, deduplica, Mermaid, lotti, geometria delle immagini e costruzione dei diagrammi.

Limiti noti

- Nessun numero dice quanta parte dell'applicazione è stata catturata**, e nessuno può dirlo: servirebbe conoscere in anticipo la risposta. Gli indicatori misurano l'ancoraggio (file citati, componenti e tabelle del parser effettivamente descritti, righe con evidenza, confidenza, chiamate non risolte). I **file muti** — quelli pieni di IF e CASE da cui non è uscita nessuna regola — sono il segnale più forte che l'applicazione sa dare su sé stessa.
- Java, COBOL e RPG restano sull'euristica**. In Python le chiamate si leggono dall'albero sintattico e in SQL l'albero le conferma; per gli altri linguaggi servirebbe un parser dedicato, che è un progetto a sé. Quelle righe restano marcate e contate per quello che sono.
- Il consolidamento vede l'inventario, non il codice**. Può collegare due nomi già trovati, non scoprire quello che nessun lotto ha visto. I lotti ora si formano seguendo le dipendenze, quindi ha molto meno da recuperare — ma il buco non è chiuso.
- Le chiavi stanno nella barra laterale**. Va bene per un prototipo; per più utenti servono variabili d'ambiente o un gestore di segreti.
- Il Word usa Arial e Courier New**, non i caratteri dell'applicazione: sono gli unici che esistono ovunque, e un documento di consegna deve apparire uguale a chiunque lo apra. Il PDF ha i font dentro il file e usa IBM Plex.
- Un documento non validato vale quanto una bozza**. L'applicazione produce una prima lettura ragionata; la firma la mette una persona.

Nota sul contratto

Il contratto JSON è alla versione **2.1**. Un JSON esportato si ricarica dal pannello «*Or resume a saved analysis*» sotto il passo 2, con le spunte e le correzioni dell'esperto; anche quelli salvati con la 2.0 — il campo `steps`, che manca, resta vuoto, e nelle analisi vecchie i diagrammi non avranno l'ordine di esecuzione dei passi.